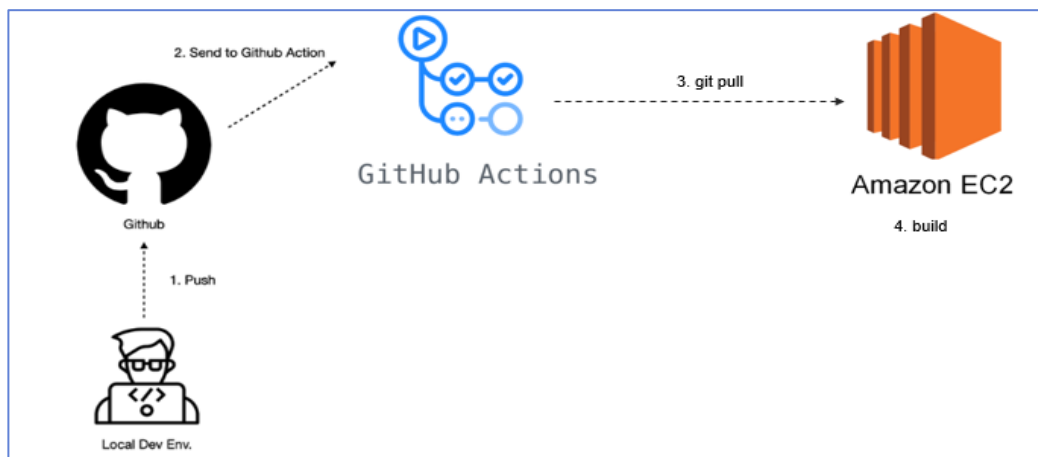


## CI/CD적용하는 방법 3가지

1. CI/CD 간단하고 빠르게 적용하는 방법(Github Actions)
2. 현업에서 초기 서비스를 구축할 때 많이 적용하는 방법
3. 서버를 여러 대 이상 구동해야 하거나 무 중단 배포가 중요한 서비스에 적용하는 방법

### 1. CI/CD 간단하고 빠르게 적용하는 방법(Github Actions)



| 장점                                                                               | 단점                                                                                                                     |
|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| (프로젝트 전체를 빌드해서 배포하는 경우와 비교해보면) Git pull 로 변경된 부분만 반영되기 때문에 속도가 빠르고, 인프라 구조가 간단하다 | (빌드 작업을 EC2에서 진행) 프로덕션 환경에서 빌드하면 시스템의 자원을 사용하므로 서버의 성능에 영향을 줄 수 있다.<br>보안상 문제 발생 소지(git pull을 위해 github 계정정보가 EC2에 저장) |

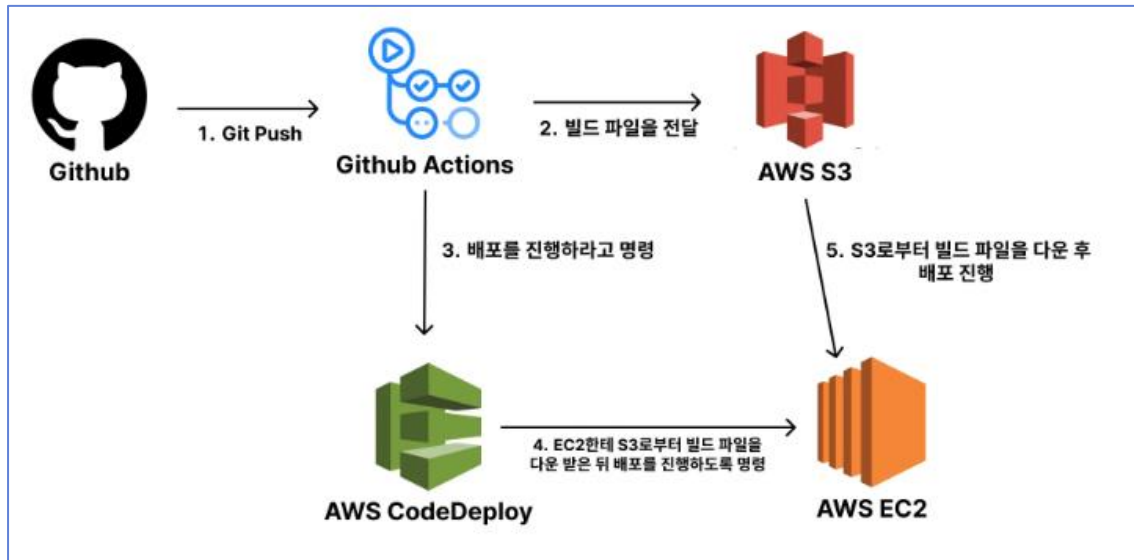
### 2. 현업에서 초기 서비스를 구축할 때 많이 적용하는 방법

(Github Actions , SCP-Secure Copy Protocol는 파일을 원격서버로 복사 전송하는 역할)



| 장점                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 단점                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>빌드 작업을 Github Actions에서 하기 때문에 운영하고 있는 서버의 성능에 영향을 거의 주지 않는다.</p> <p>CI/CD 툴로 Github Actions만 사용하기 때문에 인프라 구조가 복잡하지 않고 간단하다.</p>                                                                                                                                                                                                                                                                                                                       | <p>무중단 배포를 구현하거나 여러 EC2 인스턴스에 배포를 해야 하는 상황이라면, 직접 Github Actions에 스크립트를 작성해서 구현해야 한다. 꽤 복잡하다.</p> <p><b>무중단 배포란?</b><br/>서비스의 중단 없이 새로운 버전의 소프트웨어를 배포하는 것</p> |
| <p><b>현업에서 초기 서비스를 구축할 때 이 방법을 많이 활용한다.</b></p> <p>처음 서비스를 구현할 때는 대규모 서비스에 적합한 구조로 구현하지 않는다. 즉, 오버 엔지니어링을 하지 않는다. 확장의 필요성이 있다고 느끼는 시점에 인프라를 고도화하기 시작한다. 왜냐하면 복잡한 인프라 구조를 갖추고 관리하는 건 생각보다 아래와 같은 <b>여러 측면</b>에서 신경 쓸 게 많아지기 때문이다.</p> <ul style="list-style-type: none"> <li>• 인프라 구조를 변경할 때 시간이 많이 들어감</li> <li>• 에러가 발생했을 때 트러블 슈팅의 어려움</li> <li>• 팀원이 인프라 구조를 이해하기 어려워 함</li> <li>• 기능을 추가하거나 수정할 때 더 많은 시간이 들어감</li> <li>• 금전적인 비용이 더 많이 발생</li> </ul> |                                                                                                                                                               |

### 3. 서버를 여러 대 이상 구동해야 하거나 무 중단 배포가 중요한 서비스에 적용하는 방법 (Code Deploy)

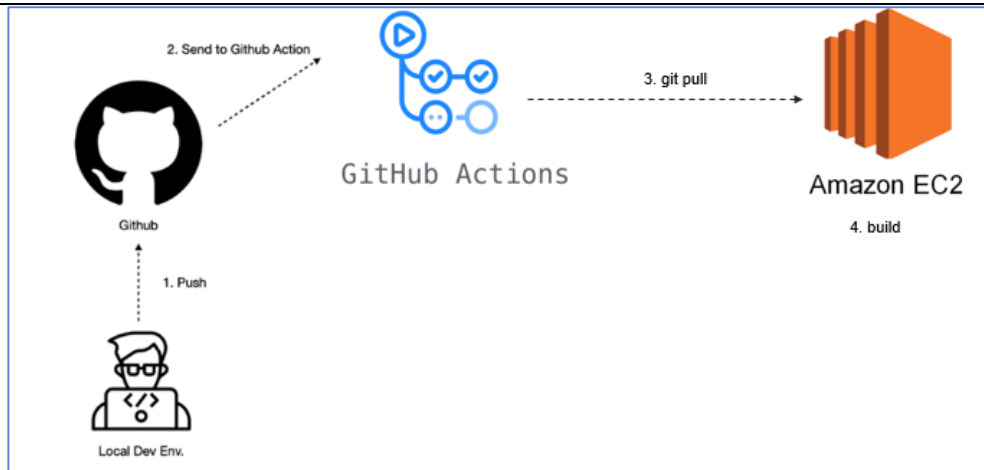


| 장점                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 단점                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <p>서버가 여러 대이더라도 쉽게 자동 배포를 구축할 수 있다.</p> <p>쉽게 무중단 배포를 적용시킬 수 있다.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                               | <p>CodeDeploy를 사용함으로써 인프라 구조가 복잡해졌다. 구조가 복잡해짐에 따라 관리 비용, 유지보수 비용, 난이도, 트러블 슈팅 어려움, 복잡도가 증가했다.</p> |
| <p><b>CodeDeploy를 사용하는 이유</b></p> <ul style="list-style-type: none"> <li>• CodeDeploy는 수많은 AWS EC2에 배포를 쉽게 할 수 있도록 도와준다.</li> <li>• CodeDeploy에 무중단 배포 기능이 내재되어 있어 손쉽게 무중단 배포를 진행할 수 있다.</li> </ul> <p><a href="https://docs.aws.amazon.com/ko_kr/codedeploy/latest/userguide/welcome.html">CodeDeploy란 무엇인가요? - AWS CodeDeploy</a></p> <p><a href="https://docs.aws.amazon.com/ko_kr/codedeploy/latest/userguide/welcome.html">https://docs.aws.amazon.com/ko_kr/codedeploy/latest/userguide/welcome.html</a></p> |                                                                                                   |

## 실습하기

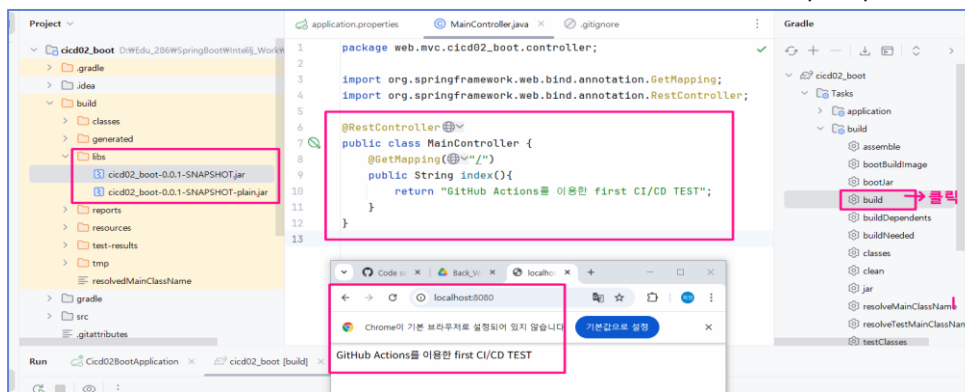
### 시나리오 01 – Github Actions를 이용한 간단한 CI/CD

☞ 전체흐름



### 작업순서

- 프로젝트 만들기 - cicd02\_boot(제공한 ex02폴더 > cicd02\_boot.zip import)



- Github에 프로젝트 push

```

KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/SpringBoot/IntelliJ_Work/cicd02_boot
$ git init
Initialized empty Git repository in D:/Edu_286/SpringBoot/IntelliJ_Work/cicd02_boot/.git/

KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/SpringBoot/IntelliJ_Work/cicd02_boot (main)
$ git add .
warning: in the working copy of '.gitattributes', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'build.gradle', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'gradle/wrapper/gradle-wrapper.properties', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'settings.gradle', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/main/java/web/mvc/cicd02_boot/Cicd02BootApplication.java', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/main/resources/application.properties', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/test/java/web/mvc/cicd02_boot/Cicd02BootApplicationTests.java', LF will be replaced by CRLF the next time Git touches it

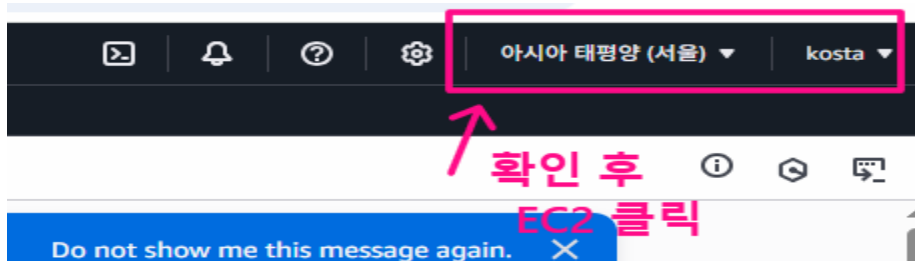
KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/SpringBoot/IntelliJ_Work/cicd02_boot (main)
$ git commit -m "first commit"
[main (root-commit) f40e14a] first commit
12 files changed, 469 insertions(+)
create mode 100644 .gitattributes
create mode 100644 .gitignore
create mode 100644 build.gradle
create mode 100644 gradle/wrapper/gradle-wrapper.jar
create mode 100644 gradle/wrapper/gradle-wrapper.properties
create mode 100644 gradlew
create mode 100644 gradlew.bat
create mode 100644 settings.gradle
create mode 100644 src/main/java/web/mvc/cicd02_boot/Cicd02BootApplication.java
create mode 100644 src/main/java/web/mvc/cicd02_boot/controller/MainController.java
create mode 100644 src/main/resources/application.properties
create mode 100644 src/test/java/web/mvc/cicd02_boot/Cicd02BootApplicationTests.java
  
```

Github에 cicd02\_boot 프로젝트를 생성 한 후 원격저장소에 push한다.

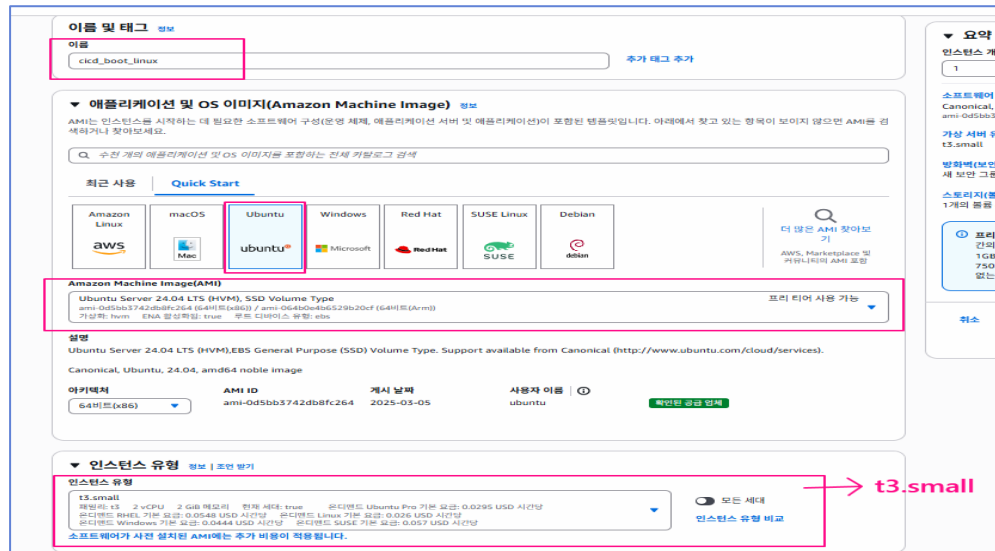
```
KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/SpringBoot/Intelilj_Work/cicd02_boot (main)
$ git remote add origin https://github.com/8253jang/cicd02_boot.git

KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/SpringBoot/Intelilj_Work/cicd02_boot (main)
$ git push -u origin main
Enumerating objects: 29, done.
Counting objects: 100% (29/29), done.
Delta compression using up to 8 threads
Compressing objects: 100% (18/18), done.
Writing objects: 100% (29/29), 45.65 KiB | 7.61 MiB/s, done.
Total 29 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/8253jang/cicd02_boot.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

- aws콘솔 로그인 > EC2 생성



## EC2 > 인스턴스 > 인스턴스 시작



## -키페어 설정

## ▼ 키 페어(로그인) 정보

키 페어를 사용하여 인스턴스에 안전하게 연결할 수 있습니다. 인스턴스를 시작하기 전에 선택한 키 페어에 대한 액세스 권한이 있는지 확인하세요.

키 페어 이름 - 필수

my-key

새 키 페어 생성

- 보안그룹 port 열기 - 8080

세부 정보

상태 및 정보

모니터링

보안

네트워킹

스토리지

태그

▼ 보안 세부 정보

IAM 역할

-

소유자 ID

 288761761976

시작 시간

Tue May 13 2025 14:31:08 GMT+0900 (한국 표준시)

보안 그룹

 sg-0752c9f90525c4d17 (launch-wizard-3)

▼ 인바운드 규칙

Q. 필터 규칙

| 이름 | 보안 그룹 규칙 ID          | 포트 범위 | 프로토콜 | 원본        | 보안 그룹                           | 설명 |
|----|----------------------|-------|------|-----------|---------------------------------|----|
| -  | sg-099e4704206fe1ecd | 8080  | TCP  | 0.0.0.0/0 | <a href="#">launch-wizard-3</a> | -  |
| -  | sg-007ae11ecc93b9c1b | 22    | TCP  | 0.0.0.0/0 | <a href="#">launch-wizard-3</a> | -  |
| -  | sg-06d89d20f41670791 | 80    | TCP  | 0.0.0.0/0 | <a href="#">launch-wizard-3</a> | -  |

- EC2에 JDK설치

```
sudo apt update
```

```
sudo apt install openjdk-17-jdk -y
```

java --version # 잘 설치됐는 지 확인

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-11-11:~$ java --version
openjdk 17.0.15 2025-04-15
OpenJDK Runtime Environment (build 17.0.15+6-Ubuntu-0ubuntu124.04)
OpenJDK 64-Bit Server VM (build 17.0.15+6-Ubuntu-0ubuntu124.04, mixed mode, sharing)
ubuntu@ip-172-31-11-11:~$
```

- EC2에 Git clone하기

```
git clone {git repository clone 주소}
```

```
ubuntu@ip-172-31-11-11:~$ ls
ubuntu@ip-172-31-11-11:~$ git clone https://github.com/8253jang/cicd02_boot.git
Cloning into 'cicd02_boot'...
remote: Enumerating objects: 29, done.
remote: Counting objects: 100% (29/29), done.
remote: Compressing objects: 100% (18/18), done.
remote: Total 29 (delta 0), reused 29 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (29/29), 45.65 KiB | 11.41 MiB/s, done.
ubuntu@ip-172-31-11-11:~$ ls
cicd02_boot
ubuntu@ip-172-31-11-11:~$
```

**참고)** Private Repository를 clone할 경우 Github의 계정과 비밀번호를 입력해야 한다. 이 때, 비밀번호는 실제 Github 계정의 비밀번호가 아닌 Github 토큰을 입력해야 한다. 토큰을 발급받는 방법은 아래 링크를 참고하자.

github 토큰 발급하기<https://velog.io/@nara7875/github-%ED%86%A0%ED%81%B0-%EB%B0%9C%EA%B8%89%ED%95%98%EA%B8%B0>

- EC2에서 clone 받은 서버가 잘 작동하는 지 확인하기

cd {프로젝트 경로}

./gradlew clean build #빌드하기

./gradlew clean build 명령어 실행 시 퍼미션 오류(permission denied)가 발생하는 경우  
(실행 권한이 없어서 생긴다)  
chmod +x ./gradlew

```
ubuntu@ip-172-31-11-11:~$ cd cicd02_boot/ 1)
ubuntu@ip-172-31-11-11:~/cicd02_boot$ ls 2)
build.gradle  gradle  gradlew  gradlew.bat  settings.gradle  src
ubuntu@ip-172-31-11-11:~/cicd02_boot$ chmod +x ./gradlew 3)
ubuntu@ip-172-31-11-11:~/cicd02_boot$ ./gradlew clean build 4)
Downloading https://services.gradle.org/distributions/gradle-8.13-bin.zip
.....10%.....20%.....30%.....40%.....50%
.....70%.....80%.....90%.....100%

Welcome to Gradle 8.13!

Here are the highlights of this release:
- Daemon JVM auto-provisioning
- Enhancements for Scala plugin and JUnit testing
- Improvements for build authors and plugin developers

For more details see https://docs.gradle.org/8.13/release-notes.html

Starting a Gradle Daemon (subsequent builds will be faster)
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader class
classpath has been appended

BUILD SUCCESSFUL in 54s
8 actionable tasks: 7 executed, 1 up-to-date
ubuntu@ip-172-31-11-11:~/cicd02_boot$ ls 5)
build  build.gradle  gradle  gradlew  gradlew.bat  settings.gradle  src
ubuntu@ip-172-31-11-11:~/cicd02_boot$
```

cd build/libs

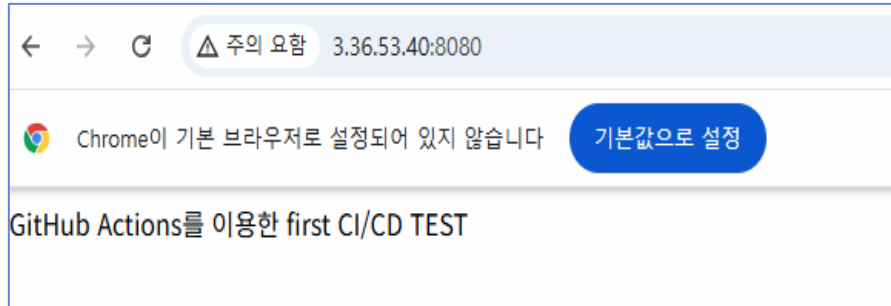
nohup java -jar cicd02\_boot-0.0.1-SNAPSHOT.jar &

```
ubuntu@ip-172-31-11-11:~/cicd02_boot$ ls
build  build.gradle  gradle  gradlew  gradlew.bat  settings.gradle  src
ubuntu@ip-172-31-11-11:~/cicd02_boot$ cd build/libs/
ubuntu@ip-172-31-11-11:~/cicd02_boot/build/libs$ ls
cicd02_boot-0.0.1-SNAPSHOT-plain.jar  cicd02_boot-0.0.1-SNAPSHOT.jar
ubuntu@ip-172-31-11-11:~/cicd02_boot/build/libs$ nohup java -jar cicd02_boot-0.0.1-SNAPSHOT.jar &
[1] 4116
ubuntu@ip-172-31-11-11:~/cicd02_boot/build/libs$ nohup: ignoring input and appending output to 'nohup.out'
```

(v3.4.5)



EC2 public IP로 브라우저로 접속해본다. (ex) ec2-public-ip:8080)



sudo lsof -i:8080 # 8080번 포트에 Spring Boot가 실행되고 있는 지 확인

```
ubuntu@ip-172-31-11-11:~/cicd02_boot/build/libs$ sudo lsof -i:8080
COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME
java 4116 ubuntu 9u IPv6 17977 0t0 TCP *:http-alt (LISTEN)
```

여기까지는 CI/CD를 하기위한 준비 작업이었다.

이제 준비가 되었으니 CI/CD를 해보자.

자동화를 적용하기 전에 동작원리를 이해하기 위해

**Git Actions를 이용하지 않는 상태에서 실제 코드가 업데이트 될 때**

**어떤 과정을 거쳐 배포가 되는지 직접 해보자.**

작업pc에서 코드작성 -> commit -> push하기 -> ec2에서 git pull받기

cd {프로젝트 경로}

git pull origin main

```
ubuntu@ip-172-31-11-11:~/cicd02_boot$ ls
build build.gradle gradle gradlew gradlew.bat settings.gradle src
ubuntu@ip-172-31-11-11:~/cicd02_boot$ git pull origin main
remote: Enumerating objects: 19, done.
remote: Counting objects: 100% (19/19), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 10 (delta 3), reused 10 (delta 3), pack-reused 0 (from 0)
Unpacking objects: 100% (10/10), 666 bytes | 333.00 KiB/s, done.
From https://github.com/8253jang/cicd02_boot
* branch      main      -> FETCH_HEAD
   f40e14a..ee793d0  main    -> origin/main
Updating f40e14a..ee793d0
Fast-forward
 src/main/java/web/mvc/cicd02_boot/controller/MainController.java | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
ubuntu@ip-172-31-11-11:~/cicd02_boot$
```

**참고) 만약, github에 프로젝트를 private로 만들었을 경우**

매번 Github 계정과 비밀번호를 치는 과정 없애기

배포를 할 때마다 Github 계정과 비밀번호를 일일이 치는 과정이 포함되어 있

으면 배포를 자동화할 수가 없다. 따라서 최초 한 번만 작성하고 그 이후에는 Github 계정과 비밀번호를 입력하지 않아도 되게 만들어보자

[Git, Pull/Push할 때 id password 묻지 않게 하기](#)

<https://pinedance.github.io/blog/2019/05/29/Git-Credential>

#### 주의)

이 방식은 ~/.git-credentials에 로그인 정보를 저장해둠으로써 github 계정과 비밀번호를 따로 묻지 않는 방식이다. 이 방식의 단점은 EC2에 접근할 수 있는 모든 사용자가 내 Github 정보를 볼 수 있다는 점이 단점이다.

#### 빌드 후 재배포하기

```
sudo lsof -i:8080 # 8080번 포트에 Spring Boot가 실행되고 있는 지 확인
```

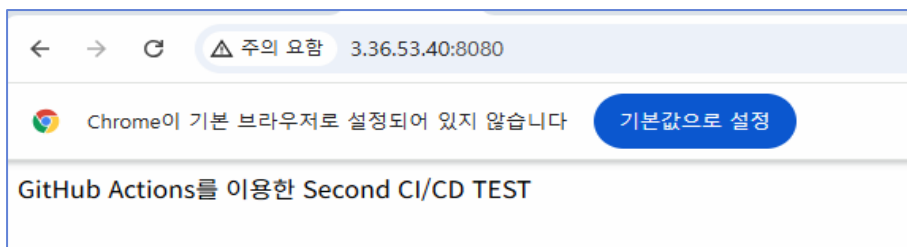
```
sudo fuser -k -n tcp 8080 # 8080번 포트에 실행되고 있는 프로세스 종료
```

```
./gradlew clean build # 권한 오류나면 chmod +x ./gradlew
```

```
cd build/libs
```

```
nohup java -jar cicd02_boot-0.0.1-SNAPSHOT.jar &
```

다시 배포된 변경 사항을 확인 할 수 있다.!



지금까지의 작업은 사람의 손을 타면서 직접 배포하는 과정이었다.

이제 지금까지의 작업을 DevOps의 문화를 정착 하게 해주는 CI/CD의 도구중의 하나인

GitHub Actions를 이용하여 수정한 코드에 대한 테스트, 통합 과정, 그리고 배포의 과정을 자동화해보도록 하자.

## Github Actions를 이용한 배포 과정 자동화

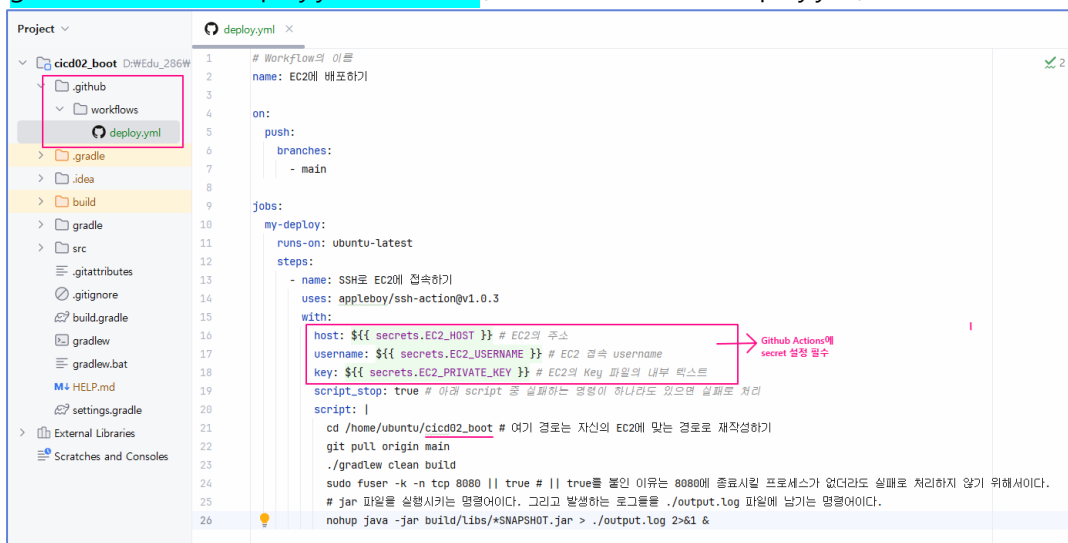
# EC2에 8080번 포트에 실행되고 있는 프로세스 종료

```
sudo fuser -k -n tcp 8080
```

# 8080번 포트에 Spring Boot가 실행되고 있는 지 확인

```
sudo lsof -i:8080
```

.github/workflows/deploy.yml 작성하기(제공파일 > ex02 > deploy.yml)



.github/workflows/deploy.yml 전체소스

# Workflow의 이름

name: EC2에 배포하기

on:

push:

branches:

- main

jobs:

my-deploy:

runs-on: ubuntu-latest

steps:

- name: SSH로 EC2에 접속하기

uses: appleboy/ssh-action@v1.0.3 # appleboy/ssh-action이라는 오픈소스 GitHub Action을 사용하여 EC2 인스턴스에 SSH로 접속

with:

host: \${ secrets.EC2\_HOST } # EC2의 주소

username: \${ secrets.EC2\_USERNAME } # EC2 접속 username

key: \${ secrets.EC2\_PRIVATE\_KEY } # EC2의 Key 파일의 내부 텍스트

script\_stop: true # 아래 script 중 실패하는 명령이 하나라도 있으면 실패로 처리

script: |

cd /home/ubuntu/cicd02\_boot # 여기 경로는 자신의 EC2에 맞는 경로로 재작성하기

git pull origin main

chmod +x ./gradlew

./gradlew clean build

sudo fuser -k -n tcp 8080 || true # || true를 붙인 이유는 8080에 종료시킬 프로세스가 없더라도 실패로 처리하지 않기 위해서이다.

```
# jar 파일을 실행시키는 명령어이다. 그리고 발생하는 로그들을 ./output.log 파일에 남기는 명령어이다.
# nohup java -jar build/libs/*SNAPSHOT.jar > ./output.log 2>&1 &
cd build/libs
nohup java -jar *SNAPSHOT.jar > ./output.log 2>&1 &
```

### nohup java -jar \*SNAPSHOT.jar > ./output.log 2>&1 &

- nohup: 터미널을 닫아도 계속 실행되게 함
- java -jar \*SNAPSHOT.jar: JAR 파일 실행
- > ./output.log: 표준 출력을 output.log 파일로 보냄
- 2>&1: 표준 에러(2)를 표준 출력(1)과 같은 곳으로 보냄 (즉, output.log로 보냄)
- &: 백그라운드에서 실행

### > ./output.log 2>&1 세부 설명

| 구성           | 의미                                  |
|--------------|-------------------------------------|
| >            | 리디렉션 연산자: **표준 출력(stdout)**을 파일로 보냄 |
| ./output.log | 현재 디렉토리에 있는 output.log 라는 파일        |
| 2>           | **표준 에러(stderr)**를 리디렉션             |
| &1           | 1번 파일 디스크립터(표준 출력)을 참조              |

즉, 2>&1 는 아래 의미입니다:

- "표준 에러(2번 파일 디스크립터)를 표준 출력(1번 파일 디스크립터)과 같은 곳으로 보내라."

### 왜 이렇게 쓰는가?

- 프로그램 실행 중 발생하는 정상 출력 + 에러 메시지를 모두 한 파일에 모으기 위해 서이다
- 로그 파일 하나만 확인하면 되니까 디버깅이나 모니터링이 편해진다.

#### 참고: 파일 디스크립터 번호

| 번호 | 의미             |
|----|----------------|
| 0  | 표준 입력 (stdin)  |
| 1  | 표준 출력 (stdout) |
| 2  | 표준 에러 (stderr) |

### Github에 Secret값 넣어주기

Settings > Security > Secrets and variables > Actions

EC2\_HOST : EC2의 공인IP 등록

EC2\_USERNAME : ubuntu

EC2\_PRIVATE\_KEY : 키페어 파일 전부 등록(공백 없는지, 앞뒤에 begin, end까지 같이 복사)

Actions secrets / New secret

Name \*

EC2\_HOST

Secret \*

3.36.53.40 — EC2 public ip

Actions secrets / New secret

Name \*

EC2\_USERNAME

Secret \*

ubuntu

Actions secrets / New secret

Name \*

EC2\_PRIVATE\_KEY

Secret \*

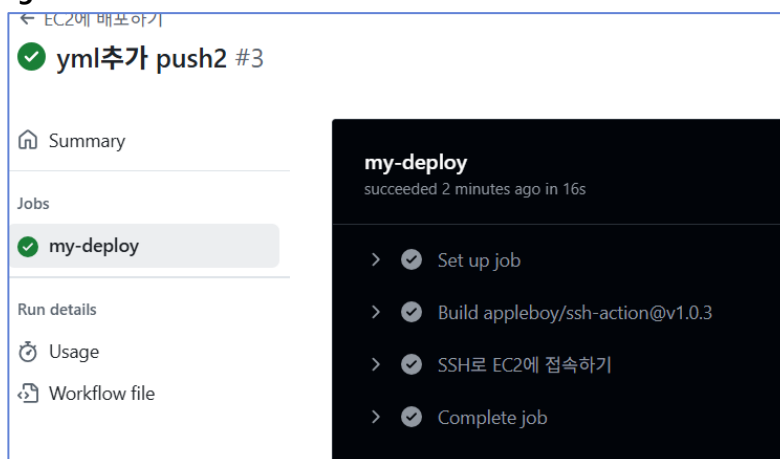
-----BEGIN RSA PRIVATE KEY-----  
MIIEowIBAAKCAQEAwdLC2O2A+InsC5AqFQSDK3TLReFlgBwcdk94cZ5qk0dKwU  
NxlVkwYITRspRJTfDapqlbVOVOQ07Ld2NkegOwHyRzcKfOKVtDF7V5wFk/c4Qq2r0  
lb0qK2HGQJ3gNxX04DeNvFB1EAofU3lPjofspkVavJM4ZF4P1ujtpP6tu668bco2  
C5p+ys3fpHG9KdCYQTe8w6tA+dFGFBQUBaV53GNlyyOSy7SK8SVrBNGljpchCh6Z  
h6SEg1w67Ohbt4gg1Z9wPNzewwDWs9UPMSxSgKXwJO5bFOGICbASlb13DHCDRKw1  
YKTTrl0ZCY99hrFPJATHW511ibiYJWF7Dq5IawlDAQABaOIBADuTn5XoDPgEIMAM  
Ujjfg6v7HCVQUdyrd9n99xpPcYaKQs5OLhwmS4lmkwbIFo0cJukW0G6s6x6hh+  
FcW6el7PSg+zSrfDq9OdrNLg7CqAd1Jlq7gnc8nD2S7HLSz7Niz9lp38/pC3UJfL  
D5g/RvX0hIL1DdjqlLw5xQRo7mwd5ArGaEWowlr11ckZBLIJvdRatBGEYJ0XALeU  
dBnzradGygtJQO5JVJtnquGvMUmlUUVKZDHyU8PA/RTQJXcGDoOIBk6EDH6/Qmos  
GjMRODLbHsDAn5Hmu3VpSn85krGH1qnkjcDGL1nw4xZVLQZK2E075eEyyeWnf/rv  
lYFFvzkCgYEA5OcNwqgFFxQGP/Xe/NP/cwAw5QDN8nn0r3trTaxlMW5QH4xGnSP  
1XIS2aXhQWkaBIA65qQPigk/v1zqgjMN/mwOkf6mAd/XC4b9UjSpqYCAWOtmmlIM7  
ALVleStwV6L6fSsXALCp5QAagzc0ezlrTyaNtXegC12ISAEBlbq5WaUCgYEA1Etm  
n4V5OaDkubFqoGFA3P9h9NCdH7VxcBLkePlylFG4UZSbegZqeyTBYfjxGrENiuOU  
M2BbsYfOCCtIT8F+HqhMjZlmrg5Q+KwLxRyHdJahAUfwlKDURq82P+SKHF9ilgHK  
yeopGuTx2Vhh3ML4EF5vc5oopH3e8dDfJHWh3M8CgYBeFkLnshx1gN5GM1FIV3uX  
vJTLzklY2hwjTT2r7aKikM9KSGR2DuO6ZGOeMqgzvazlumfB+yDNo4HYG/cGJQ1  
8cCDBgJaXuGLk7rU7HNk1yD1BafAZyoh0Vky9ntAXWETKI4SzUPhs+UVi3EuBc  
5mH1J2IXs1hjEFagwJmTKQK8gAzh+ntUT6LBFZCzu6mmDIRCve6SR4Meh7h9Ql8  
UbGmY+IHe5vFec0hQN76JV9ySRkZkf/BxC60yAsNkmaZnlar7eFV4sZDNISxBBH  
FI872M9kFy1TtsHxVgBwIPT90M2xgCXaoN3x8L84vs+PwVAOOaMxQrAgt0vDWhcz  
UDw/AoGBAOFKKeulpEppGdaUjvs4smritug6iWLVacEpPeS1qfj/fEYITHWg/HYQ  
wsrw110FkcHR5F26OPQmpnRZLMbo3BGBL+sArjP7YfW4X3bvwG0s/2xa4IRPqQ+  
oc8m6sqoGyEigaC9jZCMWBoZ8iUX+SDYJFaxgOSI/QUdgMXKXXiP  
-----END RSA PRIVATE KEY-----

Add secret



# 작업pc에서 commit > push 해본다.

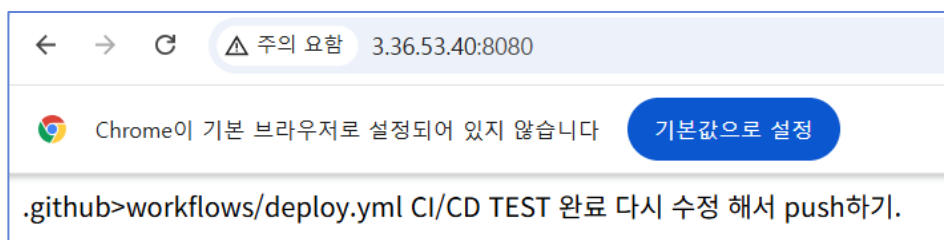
# github > Actions



# EC2 접속해서 boot 실행중인지 확인한다.

`sudo lsof -i:8080`

# 브라우저로 접속해본다.



# 작업 pc에서 변경사항을 만들고 push를 하면 자동으로 배포 완료된다!!!

.github > workflows > deploy.yml 파일의 자동화 배포 절차는 아래와 같다.

1. main 브랜치에 push 발생 시 실행

2. GitHub Actions에서 EC2로 SSH 접속
3. 최신 코드 pull 후 빌드
4. 기존 애플리케이션 종료 및 재시작

### application.properties or application.yml 파일 넣는 과정 자동화

현업 프로젝트에서는 민감한 값을 따로 application.properties 로 분리하는 경우가 많다. 민감한 값이기에 .gitignore에 추가해서 application.properties 가 버전관리 되지 않게 셋팅한다. 이 때문에 배포를 할 때 application.properties를 따로 넣어주어야 하는 귀찮은 과정이 포함된다. Github Actions를 이용하여 이 과정을 자동화시킬 수 있다.

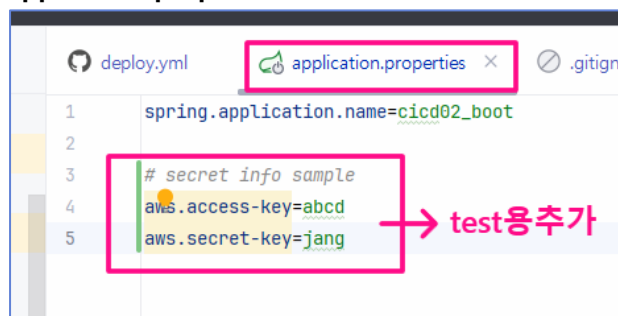
#### 작업순서

##### 1) . gitignore에 application.properties or application.yml 추가

```
39 # application.properties or application.yml 무시
40 application.*
41 |
```

application.\*

##### application.properties 수정



aws.access-key=abcd  
aws.secret-key=jang

##### 2) .github/workflows/deploy.yml 파일 수정(제공파일 > ex03\_application파일생성 폴더 참고)

```

1 # Workflow의 이름
2 name: EC2에 배포하기
3
4 on:
5   push:
6     branches:
7       - main
8
9 jobs:
10   my-deploy:
11     runs-on: ubuntu-latest
12     steps:
13       - name: SSH로 EC2에 접속하기
14         uses: appleboy/ssh-action@v1.0.3 # appleboy/ssh-action이라는 오픈소스 GitHub Action을 사용하여 EC2 인스턴스에 SSH로 접속
15         env:
16           APPLICATION_PROPERTIES: ${ secrets.APPLICATION_PROPERTIES }
17         with:
18           host: ${ secrets.EC2_HOST } # EC2의 주소
19           username: ${ secrets.EC2_USERNAME } # EC2 접속 username
20           key: ${ secrets.EC2_PRIVATE_KEY } # EC2의 Key 파일의 내부 텍스트
21           envs: APPLICATION_PROPERTIES
22           script_stop: true # 아래 script 중 실패하는 명령이 하나라도 있으면 실패로 처리
23           script: |
24             cd /home/ubuntu/cicd02_boot # 여기 경로는 자신의 EC2에 맞는 경로로 재작성하기
25             rm -rf src/main/resources/application.properties
26             git pull origin main
27             echo "$APPLICATION_PROPERTIES" > src/main/resources/application.properties
28             chmod +x ./gradlew
29             ./gradlew clean build
30             sudo fuser -k -n tcp 8080 || true # || true를 붙인 이유는 8080에 종료시킬 프로세스가 없더라도 실패로 처리하지 않기 위해서이다.
31
32             # jar 파일을 실행시키는 명령어이다. 그리고 발생하는 로그들을 ./output.log 파일에 남기는 명령어이다.
33             # nohup java -jar build/libs/*SNAPSHOT.jar > ./output.log 2>&1 &
34             cd build/libs
35             nohup java -jar *SNAPSHOT.jar > ./output.log 2>&1 &

```

### 위코드의 일 부분 해석하면

#### # GitHub Actions 내부에서 사용할 환경 변수 정의

env: APPLICATION\_PROPERTIES: \${ secrets.APPLICATION\_PROPERTIES }

#### # 위에서 정의한 환경 변수를 EC2에 전달하겠다는 뜻

envs: APPLICATION\_PROPERTIES

#### # EC2 서버 안에서 아래처럼 사용할 수 있게 됨

echo \$APPLICATION\_PROPERTIES

### github/workflows/deploy.yml 전체소스

```

# Workflow의 이름
name: EC2에 배포하기

on:
  push:
    branches:
      - main

jobs:
  my-deploy:
    runs-on: ubuntu-latest
    steps:

```



```

- name: SSH로 EC2에 접속하기
  uses: appleboy/ssh-action@v1.0.3 # appleboy/ssh-action이라는 오픈소스 GitHub Action을 사용하여 EC2 인스턴스에 SSH로 접속
  env:
    APPLICATION_PROPERTIES: ${ secrets.APPLICATION_PROPERTIES }
  with:
    host: ${ secrets.EC2_HOST } # EC2의 주소
    username: ${ secrets.EC2_USERNAME } # EC2 접속 username
    key: ${ secrets.EC2_PRIVATE_KEY } # EC2의 Key 파일의 내부 텍스트
    envs: APPLICATION_PROPERTIES
    script_stop: true # 아래 script 중 실패하는 명령이 하나라도 있으면 실패로 처리
    script: |
      cd /home/ubuntu/cicd02_boot # 여기 경로는 자신의 EC2에 맞는 경로로 재작성하기
      rm -rf src/main/resources/application.properties
      git pull origin main
      echo "$APPLICATION_PROPERTIES" > src/main/resources/application.properties
      chmod +x ./gradlew
      ./gradlew clean build
      sudo fuser -k -n tcp 8080 || true # true를 붙인 이유는 8080에 종료시킬 프로세스가 없더라도 실패로 처리하지 않기 위해서이다.

      # jar 파일을 실행시키는 명령어이다. 그리고 발생하는 로그들을 ./output.log 파일에 남기는 명령어이다.
      # nohup java -jar build/libs/*SNAPSHOT.jar > ./output.log 2>&1 &
      cd build/libs
      nohup java -jar *SNAPSHOT.jar > ./output.log 2>&1 &

```

### 3) Github에 Secret 값 넣어주기

Actions secrets / New secret

Name \*

APPLICATION\_PROPERTIES

Secret \*

spring.application.name=cicd02\_boot

# secret info sample  
aws.access-key=abcd  
aws.secret-key=jang

→ application.properties 안에 있는 모든 내용을 복사해서 넣는다.

Add secret

secret이름 : APPLICATION\_PROPERTIES

git status

```

KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/SpringBoot/Intelilj_Work/cicd02_boot (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .github/workflows/deploy.yml
        modified:   .gitignore
        modified:   src/main/resources/application.properties

no changes added to commit (use "git add" and/or "git commit -a")

KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/SpringBoot/Intelilj_Work/cicd02_boot (main)
$

```

### git 캐시 삭제

이미 add 또는 commit 해 버린 경우 git에서 추적이 되기 시작하여, gitignore에서 제외되지 않으므로 캐시삭제 필요하다.

```
git rm -r --cached .
```

Github에 resources 폴더가 push 되도록 임의의 파일 하나 만들어주기

github에 push될 때 비어 있는 폴더는 push되지 않는다.

```
src/main/resources/test.txt
```

작업pc에서 소스코드 수정 후 commit > push 해본다.

이때, application.properties는 ignore 되는지도 확인한다.

4) 실제 EC2에 application.properties or application.yml 파일도 같이 배포되었는지 확인

```

ubuntu@ip-172-31-11-11:~/cicd02_boot/src/main/resources$ ls
application.properties  test.txt
ubuntu@ip-172-31-11-11:~/cicd02_boot/src/main/resources$ cat application.properties
spring.application.name=cicd02_boot

# secret info sample
aws.access-key=abcd
aws.secret-key=jang
ubuntu@ip-172-31-11-11:~/cicd02_boot/src/main/resources$

```

← → ↺ 주의 요함 3.36.53.40:8080



Chrome이 기본 브라우저로 설정되어 있지 않습니다

기본값으로 설정

application.properties 잘 되네요~~~~

## 테스트 코드 실패하면 CI/CD 과정이 실패하는지 확인하기

`./gradlew clean build`의 과정에 테스트 코드를 실행시키는 과정이 포함되어 있다.

CI/CD를 할 때 테스트가 실패했는데 자동으로 배포가 되면 안 된다.

테스트를 실패하면 자동 배포가 진행되지 않고 중단된다.

## Spring Boot 프로젝트에 내장되어 있는 테스트 코드가 잘 작동하는 지 실행해보기

@Test 메소드안에 강제로 예외를 발생시키는 코드를 추가해보자

```
throw new RuntimeException("Fail!");
```

터미널창에서 아래 명령어 실행해본다.

```
./gradlew test
```

```

package web.mvc.cicd02_boot;

import ...

@SpringBootTest @ BongBong *
class Cicd02BootApplicationTests {

    @Test @ BongBong *
    void contextLoads() {
        //System.out.println("test종입니다.");
        throw new RuntimeException("Fail!");
    }
}

```

```

Terminal Local x
PS D:\Edu_286\SpringBoot\Intelilj_Work\cicd02_boot> ./gradlew test
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bo

> Task :test

Cicd02BootApplicationTests > contextLoads() FAILED
java.lang.RuntimeException at Cicd02BootApplicationTests.java:12

1 test completed, 1 failed

> Task :test FAILED

FAILURE: Build failed with an exception.

* What went wrong:
Execution failed for task ':test'.
> There were failing tests. See the report at: file:///D:/Edu_286/SpringBoot/Intelilj_Work/cic

```

테스트코드가 실패하는 상태로 `commit > push > github Actions` 확인 해본다.

my-deploy  
failed 1 minute ago in 18s

SSH로 EC2에 접속하기

```

33 chmod +x ./gradlew
34 ./gradlew clean build
35 sudo fuser -k -n tcp 8080 || true # || true를 붙인 이유는 8080에 종료시킬 프로세스가 없더라도 실패로 처리하지 않기 위해서이다.
36 # jar 파일을 실행시키는 명령어이다. 그리고 발생하는 로그들을 ./output.log 파일에 남기는 명령어이다.
37 # nohup java -jar build/libs/*SNAPSHOT.jar > ./output.log 2>&1 &
38 cd build/libs
39 nohup java -jar *SNAPSHOT.jar > ./output.log 2>&1 &
40 =====END=====
41 err: From https://github.com/8253jang/cicd02_boot
42 err: * branch      main      -> FETCH_HEAD
43 err: 32646a0..bd33c75  main    -> origin/main
44 out: Updating 32646a0..bd33c75
45 out: Fast-forward
46 out:  src/test/java/web/mvc/cicd02_boot/Cicd02BootApplicationTests.java | 2 ++
47 out:  1 file changed, 2 insertions(+)
48 out: > Task :clean
49 out: > Task :compileJava
50 out: > Task :processResources
51 out: > Task :classes
52 out: > Task :resolveMainClassName
53 out: > Task :bootJar
54 out: > Task :jar
55 out: > Task :assemble
56 out: > Task :compileTestJava
57 out: > Task :processTestResources NO-SOURCE
58 out: > Task :testClasses
59 err: OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
60 out: > Task :test
61 err: 1 test completed, 1 failed
62 out: Cicd02BootApplicationTests > contextLoads() FAILED
63 out:     java.lang.RuntimeException at Cicd02BootApplicationTests.java:12
64 err: FAILURE: Build failed with an exception.
65 err: * What went wrong:
66 out: > Task :test FAILED
67 err: Execution failed for task ':test'.
68 out: 8 actionable tasks: 8 executed
69 err: > There were failing tests. See the report at: file:///home/***/cicd02_boot/build/reports/tests/test/index.html
70 out: > Task :
  
```

테스트를 실패하면 자동 배포가 진행되지 않고 중단되는 것을 확인 할 수 있다.

다음 실습을 위해 ec2에 접속하여 spring boot프로젝트 종료 시키고 프로젝트 폴더를 삭제한다.

# 작업 pc에 test파일 정상동작으로 코드 변경한다.

# 8080번 포트에 실행되고 있는 프로세스 종료

```
sudo fuser -k -n tcp 8080
```

# 8080번 포트에 Spring Boot가 실행되고 있는 지 확인

```
sudo lsof -i:8080
```

# 프로젝트 폴더 삭제

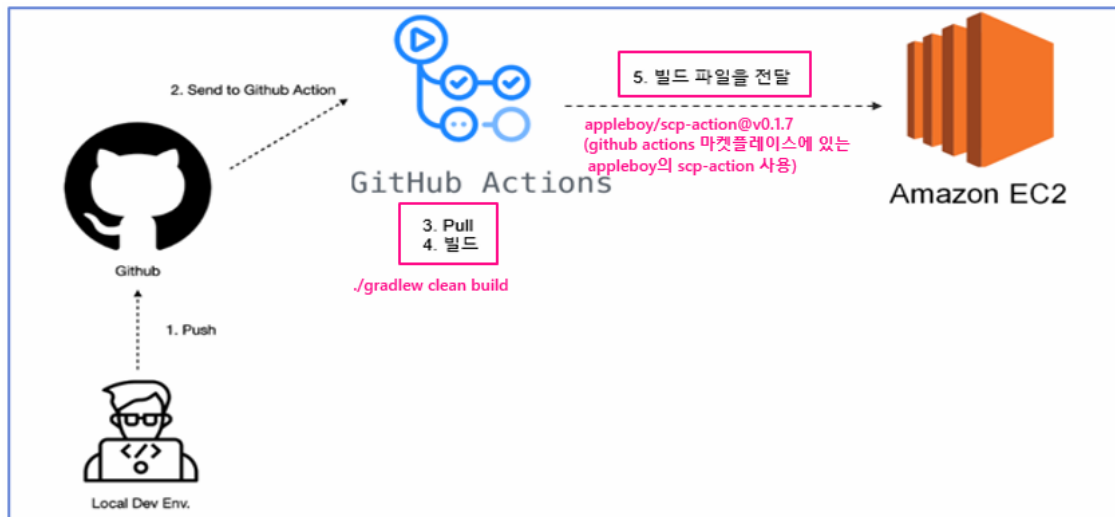
```
sudo rm -rf cicd02_boot
```

```

ubuntu@ip-172-31-11-11:~$ sudo fuser -k -n tcp 8080
8080/tcp:                  16338
ubuntu@ip-172-31-11-11:~$ sudo lsof -i:8080
ubuntu@ip-172-31-11-11:~$ ls
cicd02_boot
ubuntu@ip-172-31-11-11:~$ sudo rm -rf cicd02_boot
ubuntu@ip-172-31-11-11:~$ ls
ubuntu@ip-172-31-11-11:~$
  
```

## 시나리오 02 - 현업에서 초기 서비스를 구축할 때 많이 적용하는 방법

서버 성능에 영향을 주는 빌드작업을 운영서버에서 하는 것이 아니라 Github Actions에서 빌드작업을 하고 SCP를 이용해서 빌드파일을 EC2에 전달한다.



## .github/workflows/deploy.yml 수정내용(제공파일 > ex04\_SCP로build파일전송 폴더 참고)

```

9  jobs:
10   my-deploy:
11     runs-on: ubuntu-latest
12     steps:
13       - name: Github Repository 파일 불러오기
14         uses: actions/checkout@v4
15
16       - name: JDK 17버전 설치
17         uses: actions/setup-java@v4
18         with:
19           distribution: temurin
20           java-version: 17
21
22       - name: application.properties 파일 만들기
23         run: echo "${{ secrets.APPLICATION_PROPERTIES }}" > ./src/main/resources/application.properties
24
25       - name: gradlew 실행 권한 부여
26         run: chmod +x ./gradlew
27
28       - name: 테스트 및 빌드하기
29         run: ./gradlew clean build
30
31       - name: 빌드된 파일 이름 변경하기
32         run: mv ./build/libs/*SNAPSHOT.jar ./project.jar
33
34       - name: SCP로 EC2에 빌드된 파일 전송하기
35         uses: appleboy/scp-action@v0.1.7
36         with:
37           host: ${{ secrets.EC2_HOST }}
38           username: ${{ secrets.EC2_USERNAME }}
39           key: ${{ secrets.EC2_PRIVATE_KEY }}
40           source: project.jar
41           target: /home/ubuntu/cicd-server/tobe

```

→ deploy.yml파일에 추가된 부분

```

53 # 8. EC2에 SSH 접속하여 기존 서버 종료 → 새 프로젝트 실행
54 - name: SSH로 EC2에 접속하기
55   uses: appleboy/ssh-action@v1.0.3
56   with:
57     host: ${ secrets.EC2_HOST } # EC2 주소
58     username: ${ secrets.EC2_USERNAME } # EC2 사용자 이름
59     key: ${ secrets.EC2_PRIVATE_KEY } # SSH 개인 키
60     script_stop: true # 아래 명령 중 하나라도 실패하면 전체 실패 처리
61     script: |
62       # 기존 실행 디렉토리 삭제 후 재생성
63       rm -rf /home/ubuntu/cicd-server/current
64       mkdir /home/ubuntu/cicd-server/current
65
66       # tobe 디렉토리에서 JAR 파일을 current로 이동
67       mv /home/ubuntu/cicd-server/tobe/project.jar /home/ubuntu/cicd-server/current/project.jar
68       cd /home/ubuntu/cicd-server/current
69
70       # 기존 8080 포트 프로세스를 강제로 종료 (이미 실행 중인 서버 종료)
71       sudo fuser -k -n tcp 8080 || true
72
73       # JAR 파일 백그라운드 실행 (로그는 output.log에 기록)
74       nohup java -jar project.jar > ./output.log 2>&1 &
75
76       # 임시 저장 디렉토리(tobe)는 삭제
77       rm -rf /home/ubuntu/cicd-server/tobe
78

```

### .github/workflows/deploy.yml 수정된 전체소스

```

# 이 워크플로우의 이름
name: EC2에 배포하기

# main 브랜치에 push가 발생할 때 워크플로우 실행
on:
  push:
    branches:
      - main

jobs:
  my-deploy:
    # GitHub에서 제공하는 Ubuntu 최신 환경에서 실행
    runs-on: ubuntu-latest

    steps:
      # 1. GitHub 레포지토리 코드를 체크아웃 (가져오기)
      - name: Github Repository 파일 불러오기
        uses: actions/checkout@v4

      # 2. Java 17 설치 (Temurin JDK 사용)
      - name: JDK 17버전 설치
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 17

      # 3. application.properties 파일 생성 (GitHub Secrets에 저장된 내용을 복사)
      - name: application.properties 파일 만들기
        run: echo "${ secrets.APPLICATION_PROPERTIES }}" > ./src/main/resources/application.properties

      # 4. gradlew 파일에 실행 권한 부여 (리눅스에서 실행 가능하도록 설정)
      - name: gradlew 실행 권한 부여
        run: chmod +x ./gradlew

```

## # 5. 프로젝트 빌드 실행 (clean 후 build)

- name: 테스트 및 빌드하기  
run: ./gradlew clean build

## # 6. 빌드된 JAR 파일 이름 변경 (\*SNAPSHOT.jar → project.jar)

- name: 빌드된 파일 이름 변경하기  
run: mv ./build/libs/\*SNAPSHOT.jar ./project.jar

## # 7. 빌드된 JAR 파일을 SCP를 통해 EC2로 전송

- name: SCP로 EC2에 빌드된 파일 전송하기  
uses: appleboy/scp-action@v0.1.7  
with:  
host: \${ secrets.EC2\_HOST } # EC2 퍼블릭 IP 또는 도메인  
username: \${ secrets.EC2\_USERNAME } # 예: ubuntu  
key: \${ secrets.EC2\_PRIVATE\_KEY } # EC2에 접속 가능한 개인 키  
source: project.jar # 복사할 파일  
target: /home/ubuntu/cicd-server/tobe # 복사될 EC2 경로

## # 8. EC2에 SSH 접속하여 기존 서버 종료 → 새 프로젝트 실행

- name: SSH로 EC2에 접속하기  
uses: appleboy/ssh-action@v1.0.3  
with:  
host: \${ secrets.EC2\_HOST } # EC2 주소  
username: \${ secrets.EC2\_USERNAME } # EC2 사용자 이름  
key: \${ secrets.EC2\_PRIVATE\_KEY } # SSH 개인 키  
script\_stop: true # 아래 명령 중 하나라도 실패하면 전체 실패 처리  
script: |  
# 기존 실행 디렉토리 삭제 후 재생성  
rm -rf /home/ubuntu/cicd-server/current  
mkdir /home/ubuntu/cicd-server/current  
  
# tobe 디렉토리에서 JAR 파일을 current로 이동  
mv /home/ubuntu/cicd-server/tobe/project.jar /home/ubuntu/cicd-server/current/project.jar  
cd /home/ubuntu/cicd-server/current  
  
# 기존 8080 포트 프로세스를 강제로 종료 (이미 실행 중인 서버 종료)  
sudo fuser -k -n tcp 8080 || true  
  
# JAR 파일 백그라운드 실행 (로그는 output.log에 기록)  
nohup java -jar project.jar > ./output.log 2>&1 &  
  
# 임시 저장 디렉토리(tobe)는 삭제  
rm -rf /home/ubuntu/cicd-server/tobe

작업 pc 소스코드 수정 > commit > push > Github Actions 확인

Summary

Jobs

my-deploy

Run details

Usage

Workflow file

my-deploy

succeeded now in 57s

> Set up job

> Build appleboy/scp-action@v0.1.7

> Build appleboy/ssh-action@v1.0.3

> Github Repository 파일 불러오기

> JDK 17버전 설치

> application.properties 파일 만들기

> gradlew 실행 권한 부여

> 테스트 및 빌드하기

> 빌드된 파일 이름 변경하기

> SCP로 EC2에 빌드된 파일 전송하기

> SSH로 EC2에 접속하기

> Post JDK 17버전 설치

> Post Github Repository 파일 불러오기

> Complete job

EC2접속해서 확인

```
ubuntu@ip-172-31-11-11:~$ ls
cicd-server
ubuntu@ip-172-31-11-11:~$ cd cicd-server/
ubuntu@ip-172-31-11-11:~/cicd-server$ ls
current
ubuntu@ip-172-31-11-11:~/cicd-server$ cd current/
ubuntu@ip-172-31-11-11:~/cicd-server/current$ ls
output.log  project.jar
ubuntu@ip-172-31-11-11:~/cicd-server/current$
```

브라우저 접속해서 확인

← → ↺

⚠ 주의 요함 3.36.53.40:8080

Chrome이 기본 브라우저로 설정되어 있지 않습니다

기본값으로 설정

GitHub Actions에서 build하고 SCP로 EC2에 빌드파일전송하기 test 입니다.



## 참고) github actions 마켓플레이스

GitHub Actions에서 **uses:**에 들어가는 actions/xxx@버전 형식의 값은 **GitHub에 공개된 액션 레포지토리**를 참조하는 것이고, 그 목록은 아래의 주소에서 검색을 통해서 확인한다.

<https://docs.github.com/ko/apps/github-marketplace/github-marketplace-overview/about-github-marketplace-for-apps>

GitHub Actions Marketplace는 GitHub Actions에서 사용할 수 있는 CI/CD (Continuous Integration/Continuous Deployment) 워크플로를 구성하는 데 필요한 다양한 액션을 제공하는 온라인 상점입니다. 사용자는 여기서 자신에게 필요한 액션을 찾아 사용하거나, 직접 액션을 개발하여 공유할 수 있습니다.

The screenshot shows the GitHub Docs website. The left sidebar has a search bar and a list of navigation items under 'Apps', including '개요', 'GitHub Apps 사용', 'GitHub 앱 만들기', 'GitHub Apps 공유', 'GitHub Apps 유지 관리', 'GitHub Marketplace', and '앱용 GitHub Marketplace 정보'. The main content area is titled '앱용 GitHub Marketplace 정보' and contains the following text: '모든 사용자와 앱을 공유할 수 있는 GitHub Marketplace에 대해 알아봅니다.' followed by a paragraph explaining that GitHub Marketplace is a place where developers can find and use GitHub Actions workflows. It also mentions that users can create their own workflows and share them. At the bottom, there is a section titled 'GitHub Actions' with a link to 'GitHub Actions' and a paragraph explaining that users can create workflows and share them.

<https://github.com/marketplace?type=actions>

## Enhance your workflow with extensions

Tools from the community and partners to simplify tasks and automate processes

ssh remote

### Search results for "ssh remote"

75 results

Filter: All By: All creators Sort: Popularity

- remote\_ssh\_commands**  
Execute commands in a remote ssh server
- SSH remote run**  
Run shell scripts on remote host
- Remote SSH Publish**  
Run remote ssh commands
- Remote SSH Commands**  
Run remote ssh commands
- SSH Remote Commands**  
Executing remote ssh commands
- SSH Remote Commands - drone-ssh:1.6**  
Executing remote ssh commands

## ⚡ Quick Start

Run remote SSH commands in your workflow with minimal configuration:

```
name: Remote SSH Command
on: [push]
jobs:
  build:
    name: Build
    runs-on: ubuntu-latest
    steps:
      - name: Execute remote SSH commands using password
        uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.HOST }
          username: linuxserver.io
          password: ${ secrets.PASSWORD }
          port: ${ secrets.PORT }
          script: whoami
```

GitHub Actions에서 많이 쓰이는 대표적인 uses 예시

| 기능         | uses 예시                                               |
|------------|-------------------------------------------------------|
| 코드 체크아웃    | <code>actions/checkout@v4</code>                      |
| Node.js 설치 | <code>actions/setup-node@v3</code>                    |
| Java 설치    | <code>actions/setup-java@v3</code>                    |
| Python 설치  | <code>actions/setup-python@v4</code>                  |
| SSH 접속 배포  | <code>appleboy/ssh-action@v1.0.3</code>               |
| Docker 로그인 | <code>docker/login-action@v3</code>                   |
| AWS CLI 세팅 | <code>aws-actions/configure-aws-credentials@v2</code> |